

Chapitre Héritage

Plan du cours

Introduction

- ✓ Notion de l'héritage
- ✓ L'héritage en java
- ✓ Héritage et encapsulation
- ✓ Le mot-clé super
- ✓ La classe Object

Notion de l'héritage (1/3)

Problème

Personne	Etudiant	Employe
nom cin	nom cin moyenne	nom cin salaire
lire()	lire() reviser()	lire() travailler()



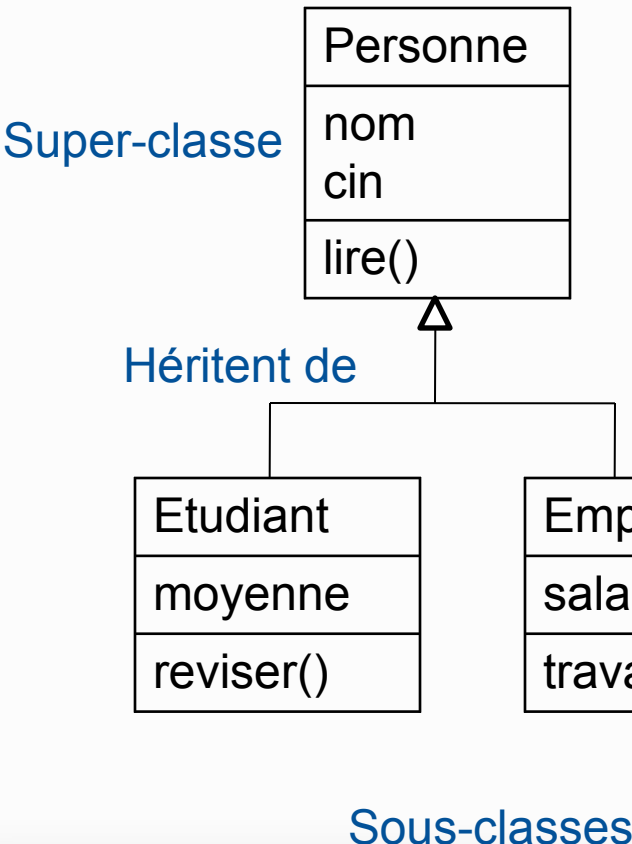
Duplication de code.



Une modification faite sur un attribut (ou méthode) commun doit être refaite dans les autres classes.

Notion de l'héritage (2/3)

Solution



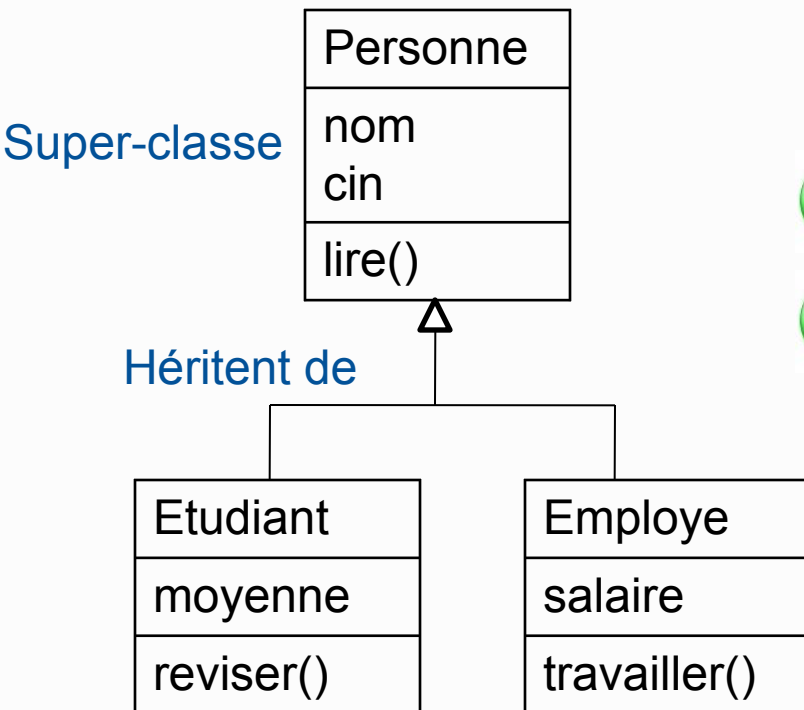
On place dans la classe mère tous les attributs et toutes les méthodes communs à toutes les classes.

Les classes filles ne comportent que les attributs ou méthodes plus spécifiques.

Les classes filles héritent automatiquement les attributs (et les méthodes) qui n'ont pas besoin d'être ré-écrits.

Notion de l'héritage (3/3)

Solution



Réutilisation du code



Un seul endroit pour faire des modifications sur les attributs (ou méthodes) en commun

Sous-classes

L'héritage en java (1/5)

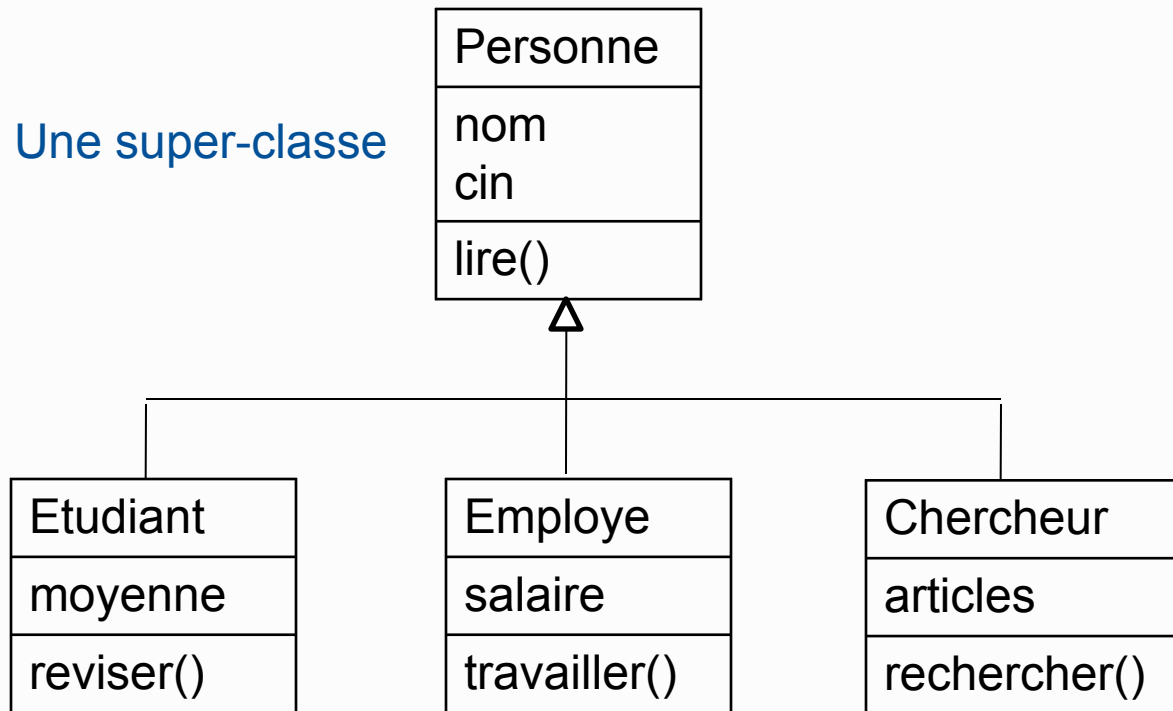
En Java, on indique qu'une classe hérite d'une autre en utilisant le mot-clé **"extends"**

```
class Personne {  
  
    String nom;  
    long cin;  
  
    public void setNom(String nom){  
        this.nom=nom;}  
  
    public String getNom(){  
        return nom;}  
  
    public void setCin(long cin){  
        this.cin=cin;}  
  
    public long getCin(){  
        return cin;}  
}
```

```
class Etudiant extends Personne {  
  
    float moyenne;  
  
    public void setMoyenne(float moyenne){  
        this.moyenne=moyenne;}  
  
    public float getMoyenne (){  
        return moyenne;}  
}
```

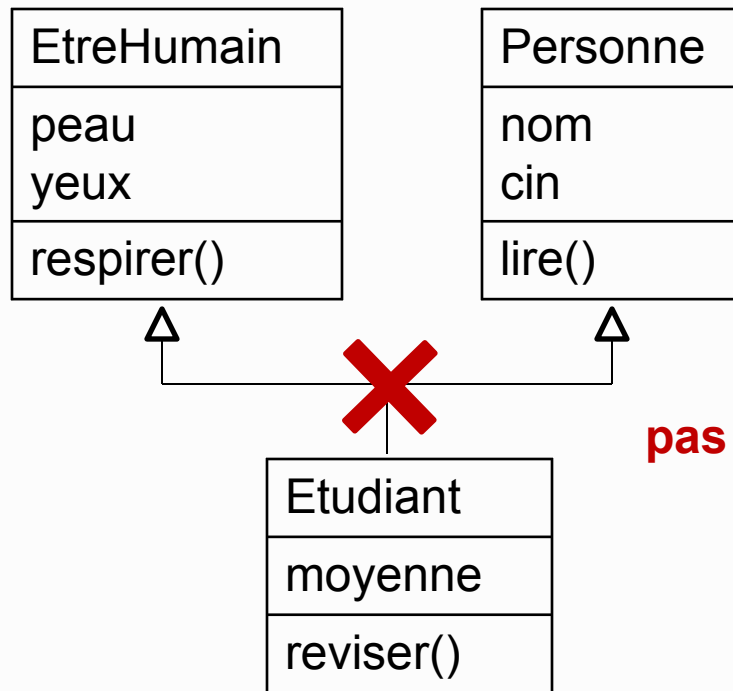
L'héritage en java (2/5)

Une classe peut avoir plusieurs sous-classes.



L'héritage en java (3/5)

Une classe ne peut avoir qu'une seule classe mère

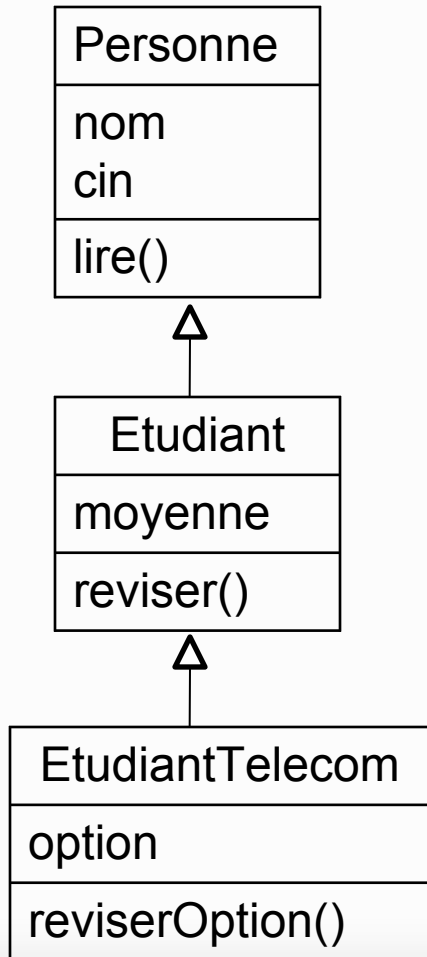


pas d'héritage multiple en java

`class Etudiant extends Personne, EtreHumain`

L'héritage en java (4/5)

Héritage en cascade



Etudiant hérite de *Personne*

EtudiantTelecom hérite de *Etudiant*

} *EtudiantTelecom*
hérite de *Personne*

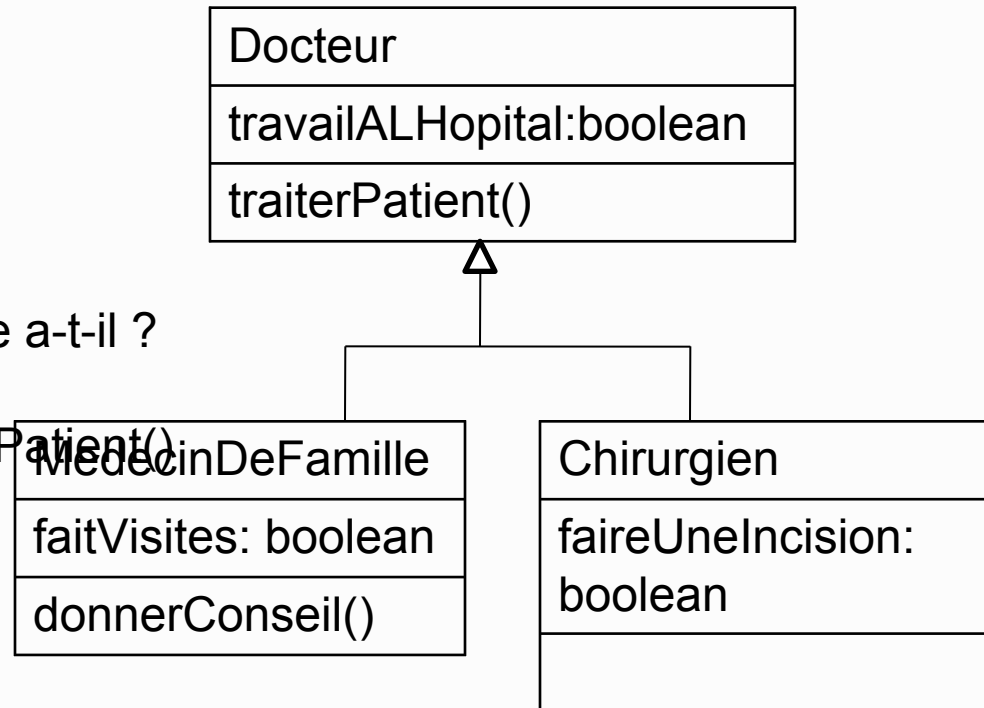
```
Etudiant etudiant=new Etudiant()
etudiant.lire(); ✓
etudiant.reviser() ✓
etudiant.reviserOption(); ✗
```

```
EtudiantTelecom etudiantTelecom=new
EtudiantTelecom() ✓
etudiantTelecom.lire(); ✓
etudiantTelecom.reviser() ✓
etudiantTelecom.reviserOption();
```

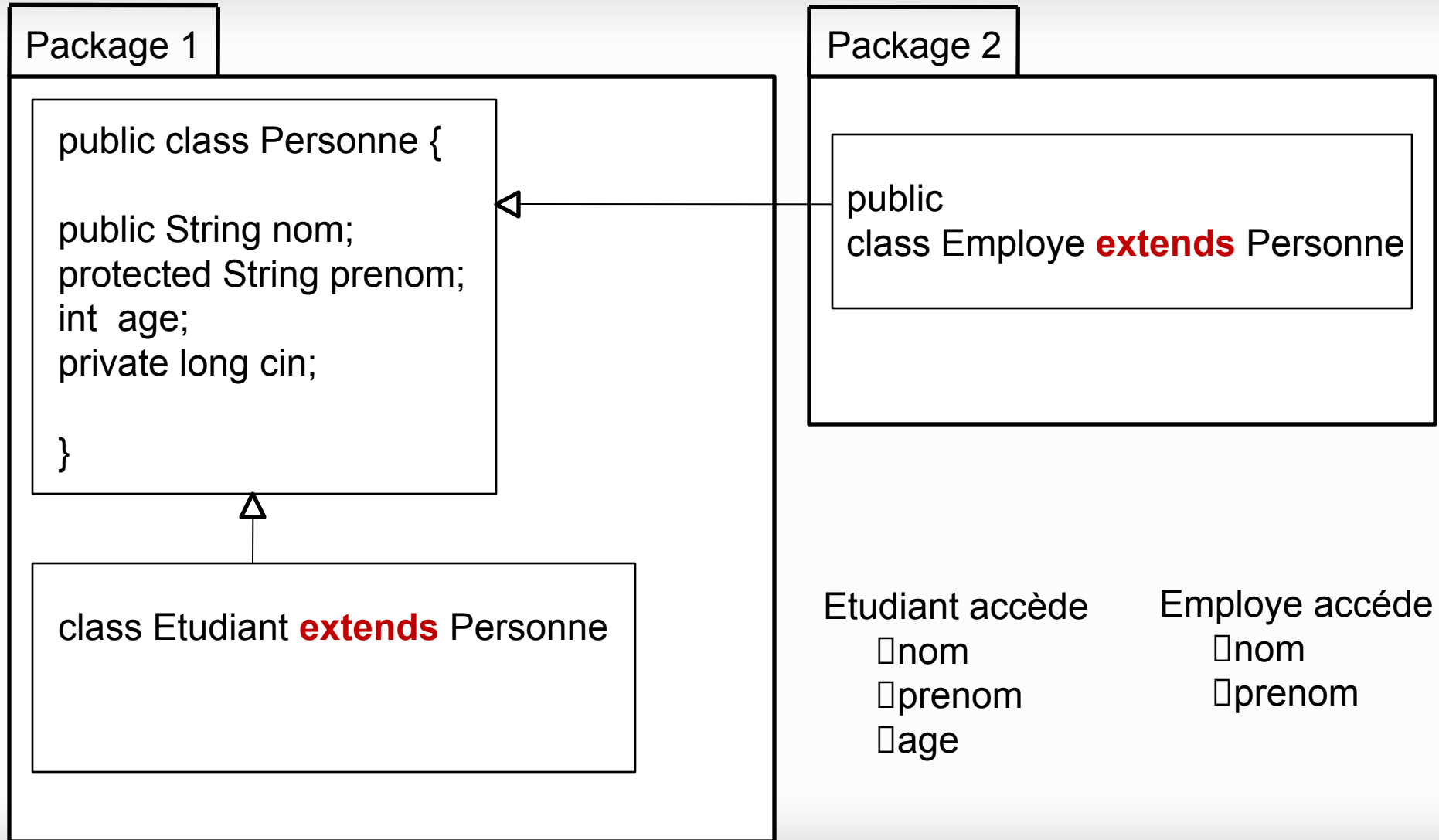
L'héritage en java (5/5)

Héritage en cascade

- Quel est le nombre de variables de Chirurgien ?
- Quel est le nombre de variables de MedecinDeFamille ?
- Combien de méthodes Docteur a-t-il ?
- Combien de méthodes MedecinDeFamille a-t-il ?
- MedecinDeFamille peut-il invoquer traiterPatient() ?
- MedecinDeFamille peut-il invoquer faireUneIncision ?



Héritage et encapsulation



Le mot-clé super

Le mot-clé *super* permet de désigner la superclasse,
super permet d'accéder aux attributs et méthodes de la super-classe

- Pour manipuler un attribut de la super-classe :

`super.cin`

- Pour manipuler une méthode de la super-classe :

`super.lire()`

- Pour faire appel au constructeur de la super-classe:

`super(nom, cin)`

Constructeur de la sous classe (1/10)

Constructeur par défaut

```
class Personne {  
  
    private String nom;  
    private long cin;  
  
}  
  
}
```

```
class Etudiant extends Personne {  
  
    private float moyenne;  
  
    public Etudiant(){  
        super();  
        moyenne=10.0f;  
    }  
  
}
```

super fait appel au constructeur par défaut de la classe mère

Constructeur de la sous classe (2/10)

Constructeur par défaut

```
class Personne {  
  
    private String nom;  
    private long cin;  
  
    public Presonne(){  
        nom="Mohamed";  
        cin=00000000;  
    }  
}
```

```
class Etudiant extends Personne {  
  
    private float moyenne;  
  
    public Etudiant(){  
        super();  
        moyenne=10.0f;  
    }  
}
```

super fait appel au constructeur sans paramètres de la classe mère

Constructeur de la sous classe (3/10)

Constructeur surchargé

```
class Personne {  
  
    private String nom;  
    private long cin;  
  
    public Presonne(String nom, long cin){  
        this.nom=nom;  
        this.cin=cin;  
    }  
}
```

```
class Etudiant extends Personne {  
  
    private float moyenne;  
  
    public Etudiant(String nom, long cin,  
        float moyenne){  
        super(nom,cin);  
        this.moyenne=moyenne;  
    }  
}
```

super fait appel au constructeur surchargé
de la classe mère

Constructeur de la sous classe (4/10)

```
public class Animal {  
  
    public int nbPattes;  
  
    public Animal(int nbPattes) {  
        this.nbPattes = nbPattes; }  
}
```

```
public class Chat extends Animal{  
}
```

Testez l'exemple suivant.

Qu'est ce que vous aurez en résultat?

Constructeur de la sous classe (5/10)

- La première instruction dans le constructeur de la sous-classe doit être l'appel au constructeur de la superclasse avec le mot clé **super**.
- Si on ne fait pas d'appel explicite au constructeur de la superclasse, c'est le **constructeur par défaut** de la superclasse qui est appelé **implicitement**.

Constructeur de la sous classe (6/10)

```
public class Animal {  
  
    public int nbPattes;  
  
    public Animal(int nbPattes) {  
        this.nbPattes = nbPattes; }  
}
```

```
public class Chat extends Animal{  
  
    public Animal(){ Constructeur par défaut  
        super();      Appel implicite à super  
    }  
}
```

Implicit super constructor
Animal() is undefined for
default constructor. Must
define an explicit
constructor

- ❑ Dans notre cas, comme nous avons écrit un constructeur surchargé pour la classe *Animal*, le constructeur *par défaut* n'existe pas.
- ❑ On a alors une erreur à la compilation lors de l'exécution de *super()*

Constructeur de la sous classe (7/10)

Solution 1

Déclarez explicitement le constructeur par défaut de la classe mère

```
public class Animal {  
  
    public int nbPattes;  
  
    public Animal(){}   
  
    public Animal(int nbPattes) {  
        this.nbPattes = nbPattes; }  
}
```

```
public class Chat extends Animal{  
}
```

Constructeur de la sous classe (8/11)

Solution 2

Faire un appel explicite au constructeur surchargé de la classe mère

```
public class Animal {  
  
    public int nbPattes;  
  
    public Animal(int nbPattes) {  
        this.nbPattes = nbPattes;  
    }  
}
```

```
public class Chat extends Animal{  
  
    public Chat(){  
        super(4);  
    }  
}
```

Constructeur de la sous classe (9/11)

Enchaînement des constructeurs

```
public class Humain{  
    public Humain() {  
        Sytem.out.println("Je suis un être humain");  
    }  
}
```

```
public class Client extends Humain {  
    public Client (){  
        Sytem.out.println("Je suis un client");  
    }  
}
```

```
public class Client Fidele extends Client {  
    public Client Fidele(){  
        Sytem.out.println("Je suis un être client fidèle");  
    }  
}
```

Constructeur de la sous classe (10/11)

Enchaînement des constructeurs

```
ClientFidele cFidele=new ClientFidele();
```

Je suis un être humain
Je suis un client
Je suis un client fidèle

Constructeur de la sous classe (11/11)

Enchaînement des constructeurs

```
public class Humain{  
    public Humain() {  
        super() ;  
        Sytem.out.println("Je suis un être humain");  
    }  
}
```

```
public class Client extends Humain {  
    public Client (){  
        super();  
        Sytem.out.println("Je suis un client");  
    }  
}
```

```
public class Client Fidele extends Client {  
    public Client Fidele(){  
        super() ;  
        Sytem.out.println("Je suis un être client fidèle");  
    }  
}
```



Appel implicite à super

Redéfinition des méthodes

```
class Personne {  
  
    public void lire() {  
        System.out.println("Je sais lire");  
    }  
  
}
```

```
class Etudiant extends Personne {  
  
    public void lire() {  
        System.out.println("Je suis un étudiant");  
        System.out.println("Je sais lire");  
    }  
  
}
```

- La classe Etudiant hérite la méthode lire() mais on veut changer son traitement

- On garde **la même signature** de la méthode lire et on change le corps



On parle alors de
redéfinition
(override)

Redéfinition avec réutilisation

```
class Personne {  
  
    public void lire() {  
        System.out.println("Je sais lire");  
    }  
  
}
```

```
class Etudiant extends Personne {  
  
    public void lire() {  
        System.out.println("Je suis un étudiant");  
        super.lire();  
    }  
  
}
```



Possibilité de réutiliser le code de la méthode héritée (super)

Surcharge

```
class Personne {  
  
    public void lire() {  
        System.out.println("Je sais lire");  
    }  
  
}
```

```
class Etudiant extends Personne {  
  
    public void lire(String titreLivres) {  
        System.out.println("Je sais lire");  
        System.out.println("J'ai lu le livre "+titreLivres);  
    }  
  
}
```

- On garde la même nom de la méthode, le même type de retour et on change les paramètres



On parle alors de
surcharge
(overload)

Surcharge vs redéfinition

```
class Personne {  
  
    public void lire() {  
        System.out.println("Je sais lire");  
    }  
}
```

surchargement

```
class Etudiant extends Personne {  
  
    public void lire(String titreLivres) {  
        System.out.println("Je sais lire");  
        System.out.println("J'ai lu le livre "+titreLivres);  
    }  
}
```

Etudiant possède deux méthodes

- lire()
- lire(String)

redéfinition

```
class Etudiant extends Personne {  
  
    public void lire() {  
        System.out.println("Je suis un étudiant");  
        super.lire();  
    }  
}
```

Etudiant possède une seule méthode

- lire()

Empêcher la redéfinition

- Une classe peut protéger une méthode afin d'éviter qu'elle ne soit redéfinie dans ses sous-classes.
- En Java, on va simplement ajouter le modificateur **final** dans la signature de la méthode.

```
public final void lire()
```

Empêcher l'héritage

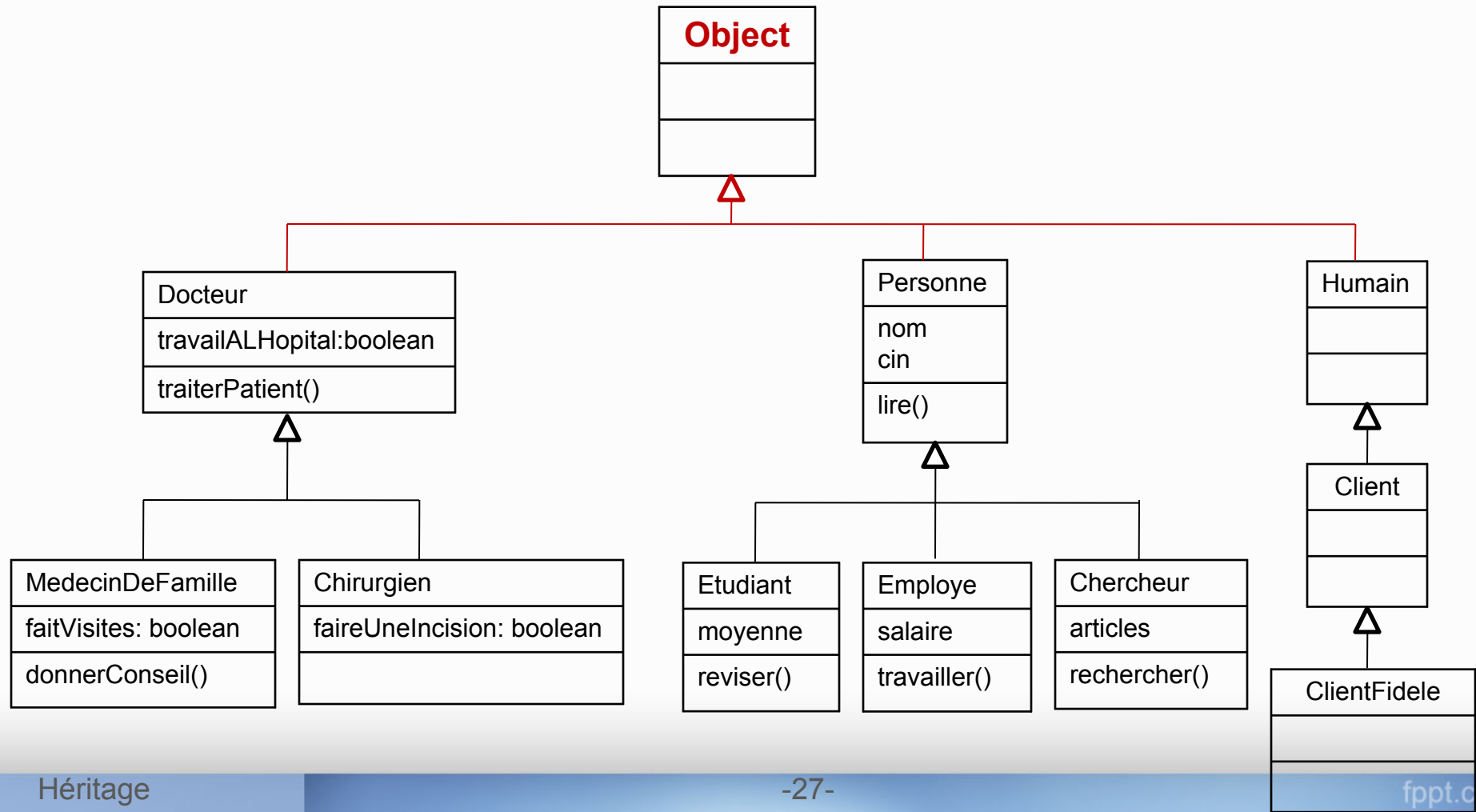
- Une classe déclarée final ne peut pas être dérivée

```
class final Personne
```

- Permet d'interdire tout héritage pour cette classe qui ne pourra pas être une classe mère
- Toutes les méthodes à l'intérieur de la classe final seront implicitement finales

La classe Object (1/2)

- Toutes les classes java héritent **implicitement** de la classe **Object**.
- On dit alors que la classe *Object* est l'*ancêtre* de toutes les classes



La classe Object (2/2)

- Méthodes importantes de la classe Object :

```
public String toString()  
public boolean equals ( Object o )  
public final Class getClass()  
public int hashCode()
```

La méthode equals (1/3)

- La méthode equals détermine si deux objets sont égaux ou non.
- Son implémentation dans la classe Object vérifie que **les références d'objets sont identiques**.
- On aura souvent besoin de redéfinir la méthode equals : **deux objets seront égaux quand ils auront le même état**.

La méthode equals (2/3)

Redéfinition de la méthode equals pour la classe Personne

Deux personnes sont égales s'ils ont le même cin et le même nom

```
public boolean equals(Object obj) {  
    if(obj instanceof Personne){  
        Personne personne = (Personne) obj;  
        if(nom==personne.nom &&cin==personne.cin)  
            { return true; }  
  
        return false;  
    }  
}
```

La méthode equals (3/3)

Erreur courante



Faire la surcharge au lieu de la redéfinition

public boolean equals(**Personne** obj) **≠** public boolean equals(**Object** obj)